

AQI Predictor Project Architecture Report

Project Overview

This project is a Karachi Air Quality Index (AQI) predictor and monitor. It collects pollution data from OpenWeather, stores raw and engineered data in MongoDB and CSV files, trains a 72-hour AQI forecasting model, and shows live AQI, pollutant readings, trends, and forecast cards in a Streamlit dashboard.

High-Level Architecture

The system works in five main steps:

1. OpenWeather API provides historical, live, and forecast air pollution data for Karachi using latitude 24.8607 and longitude 67.0011.
2. `feature_pipeline.py` fetches hourly AQI history, cleans it, builds model features, stores it in MongoDB, and also saves CSV artifacts in the `data/` folder.
3. `training_pipeline.py` loads engineered features, compares multiple machine learning algorithms, saves the best model, and writes model registry metadata.
4. GitHub Actions runs the feature pipeline hourly and the training pipeline daily or manually.
5. `app.py` runs a Streamlit dashboard that fetches live AQI/weather data, displays charts, and uses the trained model as a fallback forecast source.

Main Files And Their Role

File	Purpose
<code>app.py</code>	Streamlit dashboard for live AQI, 72-hour outlook, recent AQI trend, pollutant cards, and model fallback prediction.
<code>config.py</code>	Stores API, MongoDB, Karachi coordinates, local data path, and validation helpers.
<code>feature_pipeline.py</code>	Fetches AQI history from OpenWeather, creates raw and engineered datasets, and writes them to MongoDB/CSV.
<code>features.py</code>	Central feature engineering logic used by both feature and training pipelines.
<code>training_pipeline.py</code>	Trains and evaluates Ridge Regression, Random Forest, and XGBoost models, then saves the best one.
<code>fetch_aqi_data.py</code>	Older/helper script for fetching a fixed historical AQI range into CSV.
<code>backfill.py</code>	Uploads existing CSV data into MongoDB and rebuilds the feature store.

File	Purpose
<code>.github/workflows/train.yml</code>	Automates hourly feature updates and daily model training through GitHub Actions.
<code>data/</code>	Stores CSV datasets, trained models, feature metadata, SHAP/importance files, and latest registry pickle.

Database Architecture

The database used is MongoDB, with database name `aqi_predictor`.

Collection	Used For
<code>raw_air_quality</code>	Stores raw hourly AQI and pollutant records fetched from OpenWeather.
<code>feature_store</code>	Stores engineered feature rows used for training and dashboard history.
<code>aqi_data</code>	Legacy/backfill collection used as a fallback source for historical dashboard data.
<code>model_registry</code>	Stores model metadata such as model type, metrics, target, feature count, artifact paths, and creation time.

MongoDB is configured through the `MONGODB_URI` environment variable, and the application also keeps local CSV/pickle artifacts in `data/` so the project can still run or train when MongoDB is unavailable.

Hosting And Automation

The dashboard is a Streamlit app and can be run locally with:

```
streamlit run app.py
```

The automation is hosted on GitHub Actions through `.github/workflows/train.yml`.

Workflow Part	Schedule	What It Does
Feature pipeline	Every hour at minute 0	Fetches recent OpenWeather AQI history, updates <code>aqi_data.csv</code> , <code>aqi_features.csv</code> , and MongoDB.
Training pipeline	Daily at 00:15 UTC or manual dispatch	Trains models, saves artifacts, updates feature importance, and writes model registry metadata.

Secrets used in GitHub Actions are `OPENWEATHER_API_KEY` and `MONGODB_URI`.

Data Source

The main data source is OpenWeather Air Pollution API.

Data Type	Endpoint Usage
Historical AQI	Used by <code>feature_pipeline.py</code> and <code>fetch_aqi_data.py</code> for training data.
Live AQI	Used by <code>app.py</code> to show current Karachi air quality.
AQI forecast	Used by <code>app.py</code> as the first forecast source for the next 72 hours.
Weather data	Used by <code>app.py</code> for temperature, humidity, wind speed, and pressure sidebar values.

Model Registry

The model registry exists in two places:

Registry Location	Purpose
<code>data/model_registry_latest.pkl</code>	Local latest model metadata for quick access and version summary.
MongoDB <code>model_registry</code> collection	Persistent registry for trained model history when MongoDB is available.

The registry stores model name `aqi_72h_forecaster`, selected model type, target column, 72-hour forecast horizon, metrics, features, and artifact paths.

Algorithms Used

The training pipeline compares three algorithms and selects the one with the lowest RMSE on the final 72-hour holdout period.

Algorithm	Why It Is Used
Ridge Regression	A simple regularized linear baseline that handles many correlated pollution/time features.
Random Forest Regressor	A tree ensemble that captures non-linear relationships between pollutants and future AQI.
XGBoost Regressor	A boosted tree model that is strong for tabular prediction and complex feature interactions.

The current saved registry shows the selected model as **Ridge Regression** for target `target_aqi_72h`.

Current Model Details

Item	Value
Model name	aqi_72h_forecaster
Saved model files	data/aqi_forecast_model.pkl, data/aqi_xgb_model.pkl
Target	target_aqi_72h
Forecast horizon	72 hours
Feature count	27
Current selected algorithm	Ridge Regression
Holdout RMSE	0.5179
Holdout MAE	0.3471
Holdout R2	-0.1369
Cross-validation RMSE	1.0772

Features Used And Their Significance

Feature	Significance
aqi	Current AQI gives the model the present pollution level before forecasting future AQI.
co	Carbon monoxide helps capture traffic and combustion-related pollution.
no	Nitric oxide indicates fresh emissions, mainly from vehicles and fuel burning.
no2	Nitrogen dioxide is important for urban pollution and smog formation.
o3	Ozone captures photochemical pollution affected by sunlight and other gases.
so2	Sulfur dioxide reflects industrial or fuel-related pollution.
pm2_5	Fine particulate matter is one of the strongest AQI health indicators.
pm10	Coarse particulate matter helps represent dust and larger airborne particles.
nh3	Ammonia adds information about chemical reactions that can form secondary particles.
hour	Captures daily traffic and weather cycles.
day	Captures within-month pollution pattern changes.
month	Captures seasonal effects such as weather, humidity, and wind differences.

Feature	Significance
day_of_week	Captures weekday/weekend activity differences.
day_of_year	Represents yearly seasonal progression.
week	Captures weekly-level seasonal behavior.
timestamp	Gives a continuous time signal for long-term trend learning.
pm10_lag1	Previous-hour PM10 shows short-term persistence in dust pollution.
pm10_lag3	Three-hour PM10 lag helps catch slower pollution movement.
pm2_5_lag1	Previous-hour PM2.5 captures immediate fine-particle persistence.
aqi_lag1	Previous AQI is a direct short-term memory of air quality.
co_lag12	Twelve-hour CO lag captures delayed daily traffic/emission patterns.
pm_ratio	PM2.5/PM10 ratio helps separate fine-particle pollution from dust-heavy pollution.
pm2_5_x_no2	Interaction between PM2.5 and NO2 captures combined smog/traffic pollution effects.
so2_x_pm10	Interaction between SO2 and PM10 captures industrial-dust pollution effects.
aqi_change_rate	Measures how fast AQI is increasing or decreasing.
aqi_trend3	Three-hour AQI trend shows short-term direction.
aqi_trend6	Six-hour AQI trend shows a slightly longer pollution movement pattern.

According to the saved importance file, the strongest current signals include month, day, timestamp, day_of_year, week, co, pm2_5_x_no2, nh3, pm2_5_lag1, pm10, pm2_5, and pm_ratio.

How Prediction Works In The App

The Streamlit dashboard first tries to use OpenWeather's pollution forecast for the next 72 hours. If that forecast is not available, it loads the saved model from `data/aqi_forecast_model.pkl`, loads feature metadata from `data/feature_columns.pkl`, creates future hourly feature rows from the latest live reading, and predicts AQI values for the next 72 hours.

Important Viva Points

- The project is not only a dashboard; it has a complete data pipeline, feature store, model training pipeline, model registry, and automated workflow.
- MongoDB acts as the online storage layer, while CSV and pickle files act as local artifacts for reproducibility.
- The target is `target_aqi_72h`, meaning the model learns to predict AQI 72 hours ahead.
- The system uses time-series-aware validation through `TimeSeriesSplit`, which is better than random splitting for chronological pollution data.
- The last 72 hours are kept as a holdout test set to simulate real forecasting.
- SHAP/importance output is saved to explain which features influenced the trained model most.
- The app uses OpenWeather forecast first because it is a live external forecast, and the trained model is used as a fallback for resilience.
- GitHub Actions makes the project self-updating by refreshing data hourly and retraining daily.

Simple End-To-End Flow

```
OpenWeather API
-> feature_pipeline.py
-> MongoDB raw_air_quality + feature_store
-> training_pipeline.py
-> model artifacts + model registry
-> app.py Streamlit dashboard
-> live AQI, pollutant readings, trend charts, and 72-hour outlook
```

Conclusion

This AQI Predictor is built like a small machine learning production system. It collects real air pollution data, stores both raw and engineered forms, trains and registers forecasting models, explains feature importance, and presents live results through a Streamlit dashboard. For a viva, the main idea to explain is that the project converts hourly pollution readings into meaningful time-series features and uses them to forecast Karachi AQI up to 72 hours ahead.